Deadline-Aware Joint Random-Access Control for Dense NB-IoT Smart-Meter Networks

A Unified, Mass-Conserving Tabular Q-Learning Methodology

Proposed methodology — results and deployment claims require validation

Methodological objective and scope

This study evaluates whether a compact, network-side tabular Q-learning controller can improve deadline-compliant service for dense NB-IoT Advanced Metering Infrastructure (AMI) traffic. The candidate controller jointly selects an access-admission probability, a retry/backoff setting, and an NPRACH configuration. The scientific claim to be tested is the performance–cost trade-off of this particular controller under a declared workload, resource model, observation interface, and control-update schedule. The use of Q-learning or multiple control variables is not, by itself, claimed as novel.

The simulator, equations, reward, action registry, metrics, tables, and figures must refer to one identical implementation. All comparative curves and numerical claims must be generated from recorded runs. Synthetic proxy curves, hard-coded performance values, and fabricated variability are excluded.

A. System model and event timeline

Model one NB-IoT cell over a finite horizon H. Declare the carrier/bandwidth configuration, coverage-enhancement assumptions, random-access opportunity schedule, protocol stages represented, and the timing at which each control can take effect. The traffic generator produces periodic reports and separately parameterised event-triggered requests. Each generated message has its own request identifier; if message coalescing is used, its rule must be explicit.

At each decision epoch, process events in the following causal order: (1) create arrivals; (2) process deadlines that mature at the current time; (3) update retry timers and release eligible requests; (4) form the controller observation using only permitted measurements; (5) select a feasible action; (6) schedule the action and apply it only after its modelled signalling/effect delay; (7) execute the access attempt and any later protocol stages represented by the simulator; (8) record completions, terminal drops, and other outcomes; (9) calculate reward from realised events; and (10) construct the actual next observation/state and perform the Q update during training. The implementation and pseudocode must follow this same order.

Define the service endpoint explicitly. A singleton preamble outcome is a preamble success; it is not automatically full random-access completion or application-message delivery. Report application delivery only if the corresponding access and data-transfer stages are represented in the model.

B. Request-state model and mass conservation

Each request record contains a unique ID, generation time a_i, deadline d_i, traffic class, retry count, current status, and completion time if completed. The mutually exclusive statuses are WAITING/BARRED, BACKOFF, ACCESS-IN-PROGRESS, COMPLETED, and DROPPED. A request may transition between pending statuses, but it must never be silently removed. COMPLETED and DROPPED are terminal statuses.

$$\mathbf{A(t) = C(t) + X(t) + B(t)}$$

Here $A(t)$ is the cumulative number of generated requests by time t; $C(t)$ is the cumulative number completed; $X(t)$ is the cumulative number explicitly dropped; and $B(t)$ is the number still pending in any non-terminal status, including barred requests, requests in backoff, and procedures in progress. All four quantities are measured at the same simulation time. Check this invariant after every event batch and at the end of each run.

If finite queue storage is modelled, overflow must create an explicit drop with a recorded reason; backlog must not be silently clipped. A failed invariant invalidates the run and excludes its metrics from evaluation. This invariant establishes accounting correctness, not high performance or queue stability.

C. AMI traffic and workload generalisation

Use two separately parameterised traffic components: (i) periodic meter reports with configurable period, initial phase, and phase jitter; and (ii) event-triggered messages with declared onset, duration or arrival profile, recurrence, and affected-meter fraction. A Beta(3,4) profile may be used as one burst-shape scenario only after its mapping to request timestamps is specified; it is not assumed to represent all AMI traffic.

Evaluate feasible, near-capacity, and overloaded offered-load regimes. Population values such as 10,000, 50,000, and 100,000 are stress-test settings, not evidence that all requests can meet a chosen deadline. Establish a capacity sanity check from the implemented opportunity schedule and resource assumptions before selecting deadline targets.

Separate training traffic seeds from test seeds. Hold out event onset times, burst widths, affected fractions, and synchronisation levels for generalisation tests. Compare policies on identical test traces wherever possible, so the comparison is paired. Report all traffic parameters, seeds, test horizon, and request counts.

D. Joint action and resource-feasibility model

At decision epoch t, define the candidate action as a_t = (p_A(t), b(t), n(t)). Here p_A is the probability that an eligible request is admitted to contend; b is a supported backoff/BI setting in the model; and n identifies a feasible NPRACH configuration. Do not confuse admission probability with barring probability. If the implementation stores p_B as a barring probability, then p_A = 1 − p_B and the code, equations, plots, and labels must consistently use the chosen convention.

Construct an action registry that lists every permitted tuple, its parameter values, resource cost, effective time, and signalling/update assumptions. Call actions standards-compliant only after mapping every value and update mechanism to the applicable NB-IoT release and configuration. Without that mapping, describe the environment as an abstracted NB-IoT-inspired model.

Use a common, explicitly defined time–frequency resource accounting unit over a declared scheduling window. Let R_total be the shared resource budget, R_RACH(t) the resource allocated to random access, and R_DATA(t) the resource remaining for data. When the model assumes a shared budget, enforce R_RACH(t) + R_DATA(t) ≤ R_total and define both terms from the actual configuration-to-resource mapping. The feasible action set is A(s_t) = {a: resource_cost(a) ≤ R_total and all declared control constraints are met}.

The NPRACH–NPUSCH trade-off is a modelled constraint, not a universal proportionality rule. A larger nominal preamble/subcarrier count must not automatically be said to reduce data capacity in direct proportion unless the simulator's mapping demonstrates that relationship. If separate or non-overlapping resources are assumed, model that configuration explicitly and do not impose a zero-sum constraint that the physical model does not support.

E. Finite MDP and implementable observations

Represent the controller as a finite Markov decision process M = (S, A, P, R, γ). The transition law may be stochastic because arrivals and contention are random. The controller receives only measurements or estimates available to the network-side controller; simulator ground truth is reserved for evaluation and an explicitly labelled oracle ablation.

A candidate quantised state is s_t = (c_t, q_t, l_t), where c_t is a congestion/collision estimate, q_t is an estimate of pending or retry load, and l_t is an offered-load estimate. Candidate observations include smoothed recent attempt counts, realised successes, and estimated collision ratios. Specify the bin edges and estimator for every feature before training. The number of states is the product of the implemented bin counts; report it only after the design is fixed.

Quantisation produces a finite state space, not necessarily deterministic transitions or a guaranteed Markov state. Omitted request ages, retry timers, traffic phase, and delayed actions may make the state partially observed. Test this limitation by comparing the compact state against a richer observation and, separately, an oracle state. Label any unavailable oracle feature clearly and do not present it as deployable input.

F. Deadline-aware reward design

Use a request-centred reward based on realised events. A candidate reward for evaluation is:

$$r_t = w_{on} \cdot C_{on}(t)/K - w_{miss} \cdot M_{new}(t)/K - w_{drop} \cdot X_{new}(t)/K - w_q \cdot B_{norm}(t) - w_r \cdot R_{norm}(t)$$

C_on(t) is the number of requests newly completed on or before their deadlines during interval t. M_new(t) counts newly matured deadline misses for requests that were not already counted as dropped; X_new(t) counts newly dropped requests. B_norm(t) is a declared normalised pending-load or age-weighted backlog measure, and R_norm(t) is normalised resource use. K is a fixed reference scale selected before evaluation and shared by all policies; it must not depend on the selected action. The weights are hyperparameters tuned only on training scenarios.

The drop and deadline-miss terms represent different costs: a drop records terminal loss of service, while a matured miss records failure to meet the deadline among requests not already terminally dropped. Define event accounting so each penalty is applied at most once per corresponding event. Requests whose deadlines have not matured and that remain pending are not counted as successful or as deadline misses merely because the simulation ends; report them separately as unresolved.

Do not reward only collision reduction, and do not divide successes by a controllable quantity such as the chosen number of preambles. Such formulations can favour aggressive barring or distort comparisons between actions. Compare at least a throughput-only reward, a deadline-aware reward without resource cost, and the complete reward. Report reward-weight sensitivity and inspect whether the learned policy suppresses useful access by barring too many requests. A reward penalty is a soft optimisation objective, not proof of a hard SLA guarantee.

G. Tabular Q-learning and policy execution

During training, select a feasible action using an ε-greedy policy. Execute the action under the declared signalling delay, simulate the resulting interval, calculate reward from realised events, and construct the next state from the actual next observation. Update the Q-table using:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha_t[r_t + \gamma \max_{a' \in A(s_{t+1})} Q(s_{t+1},a') - Q(s_t,a_t)]$$

For a terminal transition, use a zero continuation value. A(s) is the feasible action set in state s. Do not use a separately constructed next-state proxy; the next state must come from the subsequent simulator observation. The selected action, the time it becomes effective, the events included in reward, and the state used for the update must be aligned.

Report the learning-rate schedule, discount factor, exploration schedule, initial Q values, episode termination rule, random seeds, and hyperparameter-selection procedure. A fixed learning rate does not by itself establish convergence. Use training data for tuning, then freeze the learned policy and evaluate it separately with ε = 0 on held-out traces.

A float32 Q-table with 150 states and 120 actions contains 18,000 entries and occupies 72,000 bytes (about 70.3 KiB) for the table alone. This is a conditional memory calculation, not a deployment measurement. Recalculate table size from the final state/action design and measure policy memory and decision latency separately from simulator overhead. A greedy scan over the action row is O(|A|); a precomputed state-to-action policy can offer constant-time lookup, but its actual runtime must be measured.

H. Baselines and ablation protocol

Maintain one baseline registry as the source of truth for implementation names, configuration, logs, tables, and figures. At minimum evaluate: (1) a tuned fixed configuration; (2) a source-faithful load-aware analytical/adaptive-barring baseline; (3) ACB-only tabular Q-learning; (4) ACB plus backoff; (5) ACB plus NPRACH configuration; and (6) the full joint controller. Tune baselines on training data and disclose any adaptations from the cited source method.

Include a DQN or other deep-RL baseline only if it is genuinely implemented, trained, tuned, and evaluated. Do not use synthetic proxy curves as algorithm evidence. Compare policies using the same traffic traces, test horizon, resource budget, observation access, terminal accounting, and declared tuning protocol. If a baseline cannot be reproduced faithfully, call it an adapted heuristic rather than the original algorithm.

I. Primary and secondary evaluation metrics

For a test horizon H, define the deadline-evaluable cohort E(H) as all requests whose deadlines have elapsed by H, together with any requests terminally dropped by H whose deadlines have not yet elapsed. This includes all requests that have already become incapable of on-time completion while excluding requests that are still pending and whose deadlines have not yet matured. For request i, define $I_{on,i}$ = 1 if it completed at or before deadline d_i, and 0 otherwise. Then:

$$P_{on}(H) = \left[\sum_{i \in E(H)} I_{on,i}\right] / |E(H)| \; ; \; P_{miss}(H) = 1 - P_{on}(H)$$

If E(H) is empty, the ratio is undefined and must be reported as not applicable, not zero. A request pending when its deadline matures, dropped without on-time completion, or completed late is a deadline miss. Requests whose deadlines have not elapsed and that remain pending at H are reported separately as unresolved, not silently counted as successes or omitted without disclosure.

Report the primary on-time completion and miss ratios alongside: cumulative completions; drops by reason; final and time-varying backlog; backlog-drain time after a finite burst; median, p95, and p99 arrival-to-completion delay among completed requests; attempt-conditioned preamble success with its exact denominator; collision rate with its event and denominator defined; resource use; and policy memory, training cost, and measured decision latency. Completed-request delay alone is subject to survivorship bias and must never be reported without request-level service outcomes.

Report energy only if a physical power/time model with units is implemented; otherwise label the quantity as an attempt or repetition proxy. Use independent training runs and held-out test traces. Report uncertainty across runs and, where possible, paired trace-level differences. A zero observed miss rate is an empirical sample outcome, not proof of zero miss probability.

J. Validation gates and reproducibility

Before comparative experiments, verify: (1) the conservation invariant after every event batch; (2) a no-arrival drain test; (3) a simple contention case against its analytical expectation under matching assumptions; (4) action feasibility and action-effective timing; (5) deadline-boundary tests, including a completion exactly at d_i; (6) reward events counted exactly once; (7) Q updates using the actual next state; and (8) reproducibility of a run from the same seed and configuration.

Log the source revision, software/runtime versions, scenario configuration, seeds, action registry, state-bin edges, hyperparameters, request-level outcomes, and raw per-run metrics. Generate all figures and tables from these logs using the same baseline registry. Inspect the final exported paper at full size and in the target two-column layout. Failed validation runs must be excluded from performance summaries or explicitly reported as failed.

Algorithm 1. Training and frozen-policy evaluation

```
Input: traffic model, request-state engine, feasible action registry A(s), state encoder, reward weights, α schedule, γ, ε
Initialise Q(s,a) for all defined states and feasible actions
For each training episode:
    Reset simulator, request ledger, timers, counters, and episode logs
    While the episode has not terminated:
        Generate arrivals and process matured deadlines
        Update retry timers; release eligible requests
        Observe permitted measurements and encode current state s
        Select feasible action a using ε-greedy policy
        Schedule/apply a according to its effective-time rule
        Simulate access and all protocol stages represented by the model
        Record completions, drops, deadline events, and resource use
        Assert A(t) = C(t) + X(t) + B(t); abort run if violated
        Compute reward from realised events in the current interval
        Observe the actual next state s′
        If transition is terminal: target = reward
        Else: target = reward + γ max over feasible a′ in A(s′) of Q(s′,a′)
        Update Q(s,a) ← Q(s,a) + α [target − Q(s,a)]
        Log state, action, reward, conservation totals, and metrics
Freeze policy; set ε = 0
Evaluate on independent held-out traces; do not update Q during testing
Report request-level service, resource use, uncertainty, and controller cost
```

K. Scope of claims

This methodology specifies a candidate experimental design; it does not establish a performance improvement before experiments are run. “Standards-compliant” requires a release-specific action and signalling map. “Real-time” requires measured latency on a stated platform. “Lightweight” requires measured memory and runtime. “Energy-efficient” requires a defensible energy model. “SLA guarantee” requires a formal guarantee or a clearly bounded empirical claim. Until those checks are complete, describe the method as a candidate controller evaluated under stated simulation assumptions.